

# AI Provider Integration Guide

Complete guide to using AI-powered semantic matching in the Topcoder Skills Recommender.

## Quick Start

### 1. Set Up OpenAI

Add your API key to `.env` file:

```
OPENAI_API_KEY=sk-your-key-here
```

### 2. Run with AI

```
# Semantic matching with OpenAI
npm start -- --matcher semantic --ai-provider openai

# Hybrid matching (recommended - combines AI with other matchers)
npm start -- --matcher hybrid --ai-provider openai

# All matchers including AI
npm start -- --matcher all --ai-provider openai
```

## Available AI Providers

### OpenAI (Implemented)

Uses `text-embedding-3-small` model for semantic similarity.

**Pros:** - High quality embeddings - Fast response times - Reliable API

**Cons:** - Costs \$0.00002 per 1K tokens (~\$0.01-\$0.05 per analysis) - Requires internet connection

#### Setup:

```
# Get API key from https://platform.openai.com/api-keys
export OPENAI_API_KEY=sk-...

# Or use .env file
echo "OPENAI_API_KEY=sk-..." >> .env

# Run
npm start -- --matcher semantic --ai-provider openai
```

### Ollama (Coming Soon)

Local AI inference with open models.

## CLI Options

# AI Provider Flags

```
--ai-provider <provider>    # AI provider: openai, ollama
--openai-key <key>          # Override OpenAI API key from env
```

## Compatible Matchers

AI provider can be used with: - `--matcher semantic` - AI-only matching - `--matcher hybrid` - AI + local matchers (recommended) - `--matcher all` - All matchers including AI

## Usage Examples

### 1. Basic AI Matching

```
npm start -- --matcher semantic --ai-provider openai
```

#### Output:

```
🚀 Topcoder Skills Recommender CLI

Configuration:
  • Matcher: semantic
  • AI Provider: openai
  • Min Confidence: 30%

Step 1: Authenticating with Github...
✅ Authenticated as: yourusername

Step 2: Loading Topcoder skills database...
✅ Loaded 10000 standardized skills

Step 3: Analyzing Github profile...
✅ Profile analyzed

Step 4: Matching skills with AI...
  Using AI Provider: OpenAI
  [Semantic Matcher] Computing semantic similarity for 100 skills...
  Processed 10/100 skills
  Processed 20/100 skills
  ...
✅ Found 15 skills above 30% confidence

🎯 Top 15 Recommended Skills:

1. JavaScript (92% confidence)
  Evidence:
  📁 [topic] AI Similarity: 0.92
  Sources: Semantic Matcher
  Rationale: AI-powered semantic match (0.920 similarity)
```

### 2. Hybrid AI + Local Matching (Recommended)

Best accuracy and cost balance:

```
npm start -- --matcher hybrid --ai-provider openai --max-repos 30
```

Combines: - LanguageMatcher (fast, reliable) - RepositoryMatcher (metadata analysis) - CommitMatcher (code patterns) - SemanticMatcher with AI (deep understanding)

### 3. Cost-Optimized AI Usage

Reduce API costs while still getting AI benefits:

```
npm start -- --matcher hybrid --ai-provider openai \
  --max-repos 20 \
  --no-deep-analysis \
  --min-confidence 50
```

### 4. Pass API Key Directly

Don't want to use .env file:

```
npm start -- --matcher semantic --ai-provider openai --openai-key sk-your-key-here
```

### 5. JSON Output for Integration

```
npm start -- --matcher semantic --ai-provider openai \
  --output json \
  --output-file results.json
```

## How It Works

### Semantic Matching Process

1. **Profile Text Construction** Programming languages: JavaScript, Python, TypeScript

Topics: react, nodejs, machine-learning

Projects: A modern web framework. ML inference library...

2. **Embedding Generation**

3. Profile text → 1536-dimensional vector

4. Each skill → 1536-dimensional vector

5. Uses OpenAI's text-embedding-3-small

6. **Similarity Computation**  $\text{cosine\_similarity} = \frac{\text{dot}(\text{profile\_vec}, \text{skill\_vec})}{(|\text{profile\_vec}| \times |\text{skill\_vec}|)}$

$\text{confidence} = \text{similarity} \times 100$

7. **Caching**

8. Embeddings cached to reduce API calls

9. Same text = cached embedding (no API cost)

### Cost Analysis

**OpenAI Pricing:** - text-embedding-3-small : \$0.00002 per 1K tokens - Average profile: ~500 tokens - 100 skills checked: ~10,000 tokens - **Total cost: ~\$0.20 per full analysis**

### Optimization Tips:

```
# Limit skills checked (default: 100)
npm start -- --matcher semantic --ai-provider openai --max-skills 50
```

```
# Limit repos analyzed (reduces profile size)
npm start -- --matcher semantic --ai-provider openai --max-repos 20
```

```
# Skip deep analysis
npm start -- --matcher semantic --ai-provider openai --no-deep-analysis
```

## Configuration

### Environment Variables

Create `.env` file:

```
# GitHub OAuth (required)
GITHUB_CLIENT_ID=your_github_client_id
GITHUB_CLIENT_SECRET=your_github_client_secret

# OpenAI (required for AI matching)
OPENAI_API_KEY=sk-your-openai-api-key

# Optional
VERBOSE=false
```

### Validation

The CLI validates: - AI provider is specified correctly - API key is available - Matcher is compatible with AI (semantic/hybrid/all)

#### Error Examples:

```
# Missing API key
npm start -- --matcher semantic --ai-provider openai
# Error: OpenAI API key required: set OPENAI_API_KEY in .env or use --openai-key

# Invalid matcher combo
npm start -- --matcher language --ai-provider openai
# Error: AI provider can only be used with semantic, hybrid, or all matchers

# Invalid provider
npm start -- --matcher semantic --ai-provider invalid
# Error: Invalid AI provider: invalid. Must be one of: openai, ollama
```

## Comparison: AI vs Non-AI

### Without AI (Keyword-Based)

```
npm start -- --matcher semantic
```

**How it works:** - Extracts keywords from profile - Matches keywords to skill index - Fast, free, but less accurate

**Best for:** - Quick analysis - Cost-sensitive scenarios - Well-defined tech stacks

### With AI (Embedding-Based)

```
npm start -- --matcher semantic --ai-provider openai
```

**How it works:** - Deep semantic understanding - Vector similarity in embedding space - Captures related concepts

**Best for:** - Nuanced skill detection - Cross-domain skills - Emerging technologies

## Side-by-Side Example

**Profile:** "Building scalable microservices with async messaging"

| Skill               | Keyword Match | AI Match | Reason                           |
|---------------------|---------------|----------|----------------------------------|
| Microservices       | 95%           | 98%      | Direct mention                   |
| Event-Driven        | 0%            | 87%      | AI understands "async messaging" |
| Distributed Systems | 0%            | 82%      | AI infers from "scalable"        |
| Message Queues      | 0%            | 91%      | AI connects "async messaging"    |

## Troubleshooting

### Error: API Key Invalid

```
# Verify key starts with 'sk-'
echo $OPENAI_API_KEY

# Test key directly
curl https://api.openai.com/v1/models \
  -H "Authorization: Bearer $OPENAI_API_KEY"
```

### Error: Rate Limited

OpenAI has rate limits. Wait or upgrade plan:

```
# Reduce requests with caching (automatic)
# Or wait 60 seconds and retry
```

### High Costs

```
# Monitor token usage in verbose mode
npm start -- --matcher semantic --ai-provider openai --verbose

# Reduce costs:
# 1. Limit skills checked
npm start -- --ai-provider openai --max-skills 50

# 2. Use hybrid (fewer AI calls)
npm start -- --matcher hybrid --ai-provider openai

# 3. Cache results
npm start -- --ai-provider openai --output-file cached.json
```

### Slow Performance

```
# AI calls are batched (10 at a time)
# First run builds cache
# Subsequent runs use cache (much faster)
```

```
# To speed up:  
npm start -- --ai-provider openai --max-skills 50 --max-repos 20
```

## Advanced Usage

### Custom Batch Size

Edit [SemanticMatcher.ts](#):

```
const batchSize = 10; // Change to 5 for slower connections
```

### Combining with Other Matchers

The `hybrid` matcher automatically weights AI results:

- Language: 1.0 (highest weight)
- Repository: 0.7
- Commit: 0.5
- **Semantic (AI): 0.6** (balanced)

### API Integration

Use JSON output for programmatic access:

```
npm start -- --ai-provider openai --output json | jq '.recommendations[]'
```

## Adding New AI Providers

See [PLUGIN\\_GUIDE.md](#) for implementing: - Ollama (local models) - Hugging Face - Anthropic Claude - Custom embedding services

## Best Practices

1. **Use Hybrid Mode** for best results
2. **Cache results** with `--output-file` for repeated analysis
3. **Set max-skills** to control costs
4. **Use verbose** to monitor API usage
5. **Combine with min-confidence** to filter results

## FAQ

**Q: Can I use AI without internet?** A: Not with OpenAI. Use Ollama (coming soon) for local inference.

**Q: How much does it cost per user?** A: ~\$0.01-\$0.20 depending on profile size and max-skills.

**Q: Is the API key safe?** A: Yes, stored in `.env` (gitignored) and never logged or transmitted except to OpenAI.

**Q: Can I use my own embeddings?** A: Yes! Implement `ISemanticProvider` interface. See [PLUGIN\\_GUIDE.md](#).

**Q: Does it work offline?** A: No for OpenAI. Use keyword-based semantic matching without `--ai-provider` flag.

## Support

For issues or questions: - GitHub Issues: [Repository](#) - Documentation: [README.md](#)